2
??
??
??

$v)$
$\delta_0 :$
$C^0(G,) \to$
$C^1(G,)$
$(\delta_0 x)_e = F_{ve} x_v - F_{ue} x_u.$
(2)
$L_0 =$
$\delta_0 \delta_0$
$x L_0 x =$
$\delta_0 x_2^2$
L_0
?
?
$G =$
$(V, E, \phi, \psi, X^{\mathrm{raw}})$
$X^{\mathrm{raw}} =$
$\{x_v^{\mathrm{raw}}\}_{v \in V}$
$\phi :$
$V \to$
$\mathcal{T}$
$\psi :$
$E \to$
$\mathcal{R}$
v
$x_v^{\mathrm{raw}} \in^{d_{\phi(v)}}$
$t \in$
$\mathcal{T}$
$V_t =$
$\{v \in$
$V :$
$\phi(v) =$
$t\}$
Σ
$\sigma =$
$(t_0, r_1, t_1, \ldots, r_\ell, t_\ell)$
$f =$
$(v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$
$\phi(v_i) =$
t_i
$\psi(e_i) =$
r_i
Σ
$\mathcal{I}_\sigma$
σ
$\mathcal{I}_\Sigma =$
$\bigcup_{\sigma \in \Sigma} \mathcal{I}_\sigma$
$f \in$
$\mathcal{I}_\Sigma$
G
??
Data
In-
put
Se-
lec-
tion
Learn-
ing
$framework_overview.pngOverviewof.Typedgraphdataenteratraining - onlyselectionstage; selectedschemasandweighted pathir$
x_v
$\widehat{y}_v$
$m \in^{4|\mathcal{R}|}$
$x_v = x_v^{\mathrm{raw}} m.$
(3)
be-
fore
$h_v^{(0)} = Dropout\big[GELU\big(LN(W_{\phi(v)} x_v + b_{\phi(v)}) + q_{\phi(v)}\big)\big] \in^d,$
(4)
W_t, b_t
q_t
$h_v^{(0)}$
$x^{(0)} =$
$h^{(0)}$
r
$z_r \in^{d_r}$
$e =$
(urv)
$F_{ue}, F_{ve} \in^{d \times d}$
$h_u^{(0)}$
$h_v^{(0)}$
z_r
$F_{ve} = \lambda_{v,e} Q_{v,e}, Q_{v,e} Q_{v,e} = I, \lambda_{v,e} > 0,$
(5)
F_{ue}

H2S-NN: Heterophilic Heterogeneous Neural Networks with Adaptive Meta-Path Selection

Anonymous Author(s)

Abstract

Heterogeneous graphs contain many node and relation types, and learning on them raises two problems that are usually handled separately but really occur together. Different relation types carry different meanings, so a single shared update rule can blur information that should stay type-aware; and many of these relations are heterophilic, where connected nodes need not share a label or similar features, so a model that assumes neighbors agree can discard the very signal a relation carries. Meta-paths, typed relations such as author-paper-author, are the standard way to expose multi-hop semantics, but existing methods collapse each into a node-to-node view and lose the path structure a heterophily-aware model could act on. We introduce H2S-NN, which keeps both problems in view. It compares each edge's endpoints in a shared space, so that a relation-aware disagreement rather than a raw feature gap drives learning, and it retains selected meta-path instances as higher-order structure instead of reducing them to endpoint pairs. To avoid hand-picking meta-paths, AMPSS learns which schemas and instances matter jointly with the task, while SheafConvPool pools the result for graph classification. Across benchmarks, H2S-NN leads on node and graph classification and stays competitive on link prediction; ablations confirm that each component contributes.

CCS Concepts

• **Computing methodologies** → **Neural networks**.

Keywords

heterogeneous graph neural networks, heterophily, sheaf neural networks, meta-path discovery, graph pooling

ACM Reference Format:

Anonymous Author(s). 2026. H2S-NN: Heterophilic Heterogeneous Neural Networks with Adaptive Meta-Path Selection. In *Proceedings of ACM Conference (Conference acronym 'XX)*. ACM, New York, NY, USA, ?? pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Heterogeneous graphs are a common representation for modeling complex relational structures, such as citation networks [?] or social interactions [?]. Unlike homogeneous graphs, heterogeneous graphs contain multiple types of nodes and edges wwith distinct semantic roles [?]. Moreover, many graphs are also heterophilic,

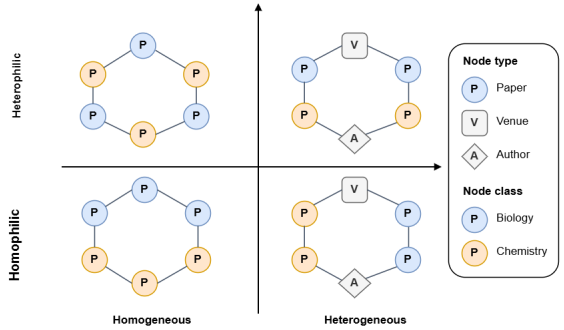


Figure 1: An example of heterogeneous heterophilic graph. The graph contains multiple types of nodes and edges, and connected nodes may have different properties.

where nodes may have different properties. A graph can therefore be both heterogeneous and heterophilic, resulting in a much more complex learning task called heterogeneous heterophilic graph learning, as shown in Figure **??**. Furthermore, modeling graphs under the simplifying assumptions of homogeneity or homophily can obscure important structural and relational patterns. This mismatch provides the model with an incomplete representation of the original graph and may consequently degrade predictive performance [?].

Many existing methods address either heterogeneous graphs without explicitly considering heterophily [? ? ? ? ? ?] or heterophilic graphs without modeling heterogeneous node and edge types [? ? ? ?]. Recent methods have begun to address heterogeneous heterophilic graphs through meta-path-based latent subgraphs [? ? ? ? ? ?], graph rewiring [?], graph transformers [?], and sheaf-based mappings [?]. Among these directions, meta-paths can express information that is not contained in either endpoint alone [?]. For instance, an `Author-Paper-Author` meta-path describes a co-authorship relation in citation networks, as shown in Figure **??**. However, existing methods either ignore meta-paths completely, use meta-paths as semantic views, filtered edge sets, or aggregation channels rather than retaining each concrete path instance as part of the propagation structure. For example, an `A-P-A` meta-path can connect two authors through a shared paper, but after it is converted into an author-author view, the shared paper and the two authorship edges no longer directly shape the propagation operator. This is limiting under heterophily, because whether information should be reinforced or contrasted can depend on the relations along the path, not only on the endpoint connection. This can weaken locality and higher-order information from meta-paths as a part of graph structure.

Manual meta-path specification creates a second bottleneck. Hand-crafted meta-paths are difficult in complex networks, and some useful paths might be ignored [? ?]. Moreover, datasets often

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, City, Country

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2026/MM
https://doi.org/XXXXXXX.XXXXXXX

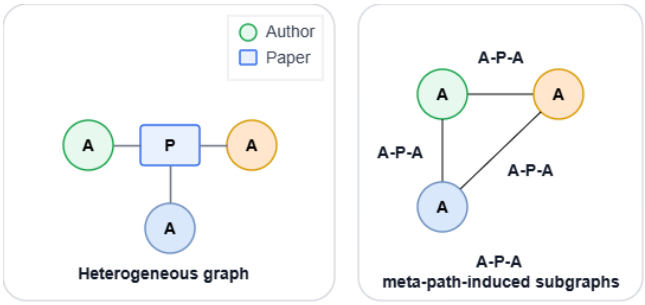


Figure 2: Illustration of meta-path-based graph transformation. The original heterogeneous graph contains author and paper nodes connected by authorship relations. Under the A-P-A meta-path, two authors are connected when they share a common paper, producing an author-level meta-path-induced subgraph.

lack such predefined meta-paths. There exists multiple methods for finding useful meta-paths automatically, but they are usually computationally expensive. Some methods reduce this burden through progressive sampling [**?**], searching algorithm [**? ? ? ?**], or reinforcement learning [**?**]. Although automatic meta-path search reduces reliance on expert knowledge, most methods still select or weight meta-paths at the schema level. This is too coarse: a PAP schema may be useful overall, while some concrete PAP instances connect unrelated papers and introduce noise. Schema-level selection must therefore keep noisy instances with useful ones or discard the entire schema. In addition, many search methods select schemas before training the prediction model, so schema choice and instance relevance cannot be refined jointly by the task objective.

To address these problems, we propose H2S-NN, a heterogeneous heterophilic sheaf-Hodge neural network for heterogeneous heterophilic graphs that treats meta-path instances as higher-order. We firstly propose a method to model a heterogeneous heterophilic graph with meta-paths as a 2D-cellular graph. Then, we learn both structure via sheaf restriction maps and diffusion, preserving locality which causes heterophily and heterogeneous from graph structure. We further introduce AMPSS, a two-stage schema-to-cell meta-path selection mechanism that learns soft weights for both candidate schemas and individual path instances, identifying which relational patterns matter and which concrete occurrences are informative and can be learned during training. Furthermore, for graph classification task, it requires converting node and edge sheaf states into a single graph-level representation. However, because H2S-NN uses local vector spaces, this readout should not depend on an arbitrary choice of local basis. For this reason, we introduce SheafConvPool, a gauge-invariant readout that pools attention-weighted mean and covariance statistics from Gram descriptors of node and edge sheaf states. This produces graph-level representations invariant to H2S-NN's orthogonal restriction maps by construction (§ **??**).

Together, these components cover three stages of the pipeline: AMPSS selects the typed higher-order support, the sheaf-Hodge model learns diffusion over node and edge ranks, and SheafConvPool produces a gauge-invariant graph-level summary of that diffusion when a graph-level prediction is required.

The contributions of this work are as follows.

(1) We introduce H2S-NN, a heterogeneous sheaf-Hodge framework that retains concrete typed path instances as a face-like component on our proposed 2D-cellular graphs, resulting in a learned graph structure that preserves locality without breaking heterophily on heterogeneous graphs.
(2) We propose AMPSS, a two-stage schema-to-cell selection mechanism that combines training-only candidate preselection with differentiable schema- and instance-level refinement inside the upper edge Laplacian. This allows H2S-NN to learn which concrete occurrences of a schema are useful for the task.
(3) For graph classification, we introduce SheafConvPool, a gauge-invariant readout that scores per-node and per-edge Gram descriptors with learned attention and pools their attention-weighted mean and covariance, producing a graph-level representation that is invariant to H2S-NN's orthogonal restriction maps by construction.

To this end, we evaluate the framework across node classification, graph classification, and heterogeneous link prediction. As a result, H2S-NN achieves the best overall performance across all tasks including node classification, graph classification, and heterogeneous link prediction.

2 Preliminaries

2.1 Heterogeneous graphs

A heterogeneous graph is a tuple $G = (V, E, \phi, \psi)$, where V and E are the node and edge sets, $\phi : V \to \mathcal{T}$ assigns each node a type, and $\psi : E \to \mathcal{R}$ assigns each edge a relation type. The realized type sets are $\mathcal{T} = \{\phi(v) : v \in V\}$ and $\mathcal{R} = \{\psi(e) : e \in E\}$. The graph is heterogeneous when it contains more than one node type or more than one relation type, i.e., $|\mathcal{T}| > 1$ or $|\mathcal{R}| > 1$; it reduces to a homogeneous graph when $|\mathcal{T}| = |\mathcal{R}| = 1$ [**?**].

2.2 Homophily and Heterophily

Homophily describes the tendency of connected nodes to share the same label. For a graph $G' = (V', E')$ whose endpoint labels are comparable, its edge homophily is

$$H_{\text{edge}}(G') = \frac{1}{|E'|} \sum_{(u,v)\in E'} \mathbf{1}[y_u = y_v], \tag{26}$$

where y_u is the label of node u and $\mathbf{1}[\cdot]$ is the indicator function [**?**]. Heterophily describes the opposite tendency: adjacent nodes are more likely to have different labels. Larger homophily values indicate stronger label agreement. For heterogeneous graphs, we compute homophily on each meta-path-induced homogeneous subgraph and average across the considered meta-paths [**?**]:

$$\mathcal{H} = \frac{1}{|\mathcal{P}|} \sum_{p\in\mathcal{P}} H_{\text{edge}}(G'_p), \tag{27}$$

where $\mathcal{P}$ denotes the set of all considered meta-paths and G'_p is the homogeneous subgraph induced by meta-path p.

2.3 Meta-path

A meta-path schema of length ℓ is a typed relation sequence

$$\sigma : t_0 \xrightarrow{r_1} t_1 \xrightarrow{r_2} \cdots \xrightarrow{r_\ell} t_\ell, \qquad t_i \in \mathcal{T},\ r_i \in \mathcal{R}, \tag{28}$$

which specifies a composite relation between its endpoint types [**?**]. For each relation r, let $A_r \in \{0,1\}^{|V|\times|V|}$ be its adjacency matrix, with $(A_r)_{uv} = 1$ when an edge of type r connects u to v. The schema induces the walk-count matrix

$$A_\sigma = A_{r_1} A_{r_2} \cdots A_{r_\ell}, \tag{29}$$

where $(A_\sigma)_{uv}$ counts the concrete walks from u to v that follow σ. When $t_0 = t_\ell$, thresholding A_σ produces a homogeneous endpoint graph on which Eq. (**??**) can be evaluated. For example, the paper-author-paper schema has $A_{\mathrm{PAP}} = A_{\mathrm{PA}} A_{\mathrm{AP}}$ and connects papers that share an author. A concrete path instance is an ordered sequence $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$ satisfying $\phi(v_i) = t_i$ and $\psi(e_i) = r_i$. Thus, σ describes a type-level pattern, whereas f identifies one occurrence of that pattern in the observed graph.

3 Methodology

3.1 Problem Formulation

We consider a directed heterogeneous graph $G = (V, E, \phi, \psi, X^{\mathrm{raw}})$, where $X^{\mathrm{raw}} = \{x_v^{\mathrm{raw}}\}_{v\in V}$, and $\phi : V \to \mathcal{T}$ and $\psi : E \to \mathcal{R}$ assign node and relation types. A node v has a raw feature vector $x_v^{\mathrm{raw}} \in \mathbb{R}^{d_{\phi(v)}}$, so different node types may use different input dimensions. For each $t \in \mathcal{T}$, let $V_t = \{v \in V : \phi(v) = t\}$. We make no homophily assumption: adjacent nodes may have different labels or dissimilar features. Each heterogeneous graph has a set of meta-path schemas Σ. A meta-path schema $\sigma = (t_0, r_1, t_1, \ldots, r_\ell, t_\ell)$ specifies a valid typed relation sequence, whereas $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$ denotes one concrete instance satisfying $\phi(v_i) = t_i$ and $\psi(e_i) = r_i$. Let Σ be the schema pool supplied manually or selected by AMPSS, let $\mathcal{I}_\sigma$ be the instances of schema σ, and let $\mathcal{I}_\Sigma = \bigcup_{\sigma\in\Sigma} \mathcal{I}_\sigma$. The model retains each $f \in \mathcal{I}_\Sigma$ as a face-like support over its ordered path edges. Given G and this support, the shared learning problem is to infer node-to-edge and edge-to-face maps together with node and edge cochains that serve the following task settings.

3.2 Framework Overview

We introduce H2S-NN, a Heterophilic Heterogeneous Sheaf-Hodge Neural Network for supervised graph neural networks. As illustrated in Figure **??**, H2S-NN is organized into four stages. *Data Input* prepares typed features, relations, and training-only diagnostics. *AMPSS Selection* constructs a compact schema pool and refines the importance of its concrete instances. *H2S-NN Learning* uses the selected paths to learn node-edge-face geometry and diffuse node and edge cochains. Finally, a task-specific readout predicts target-node labels, candidate links, or graph labels. These tasks share the same encoder and operators: node classification reads selected target nodes, link prediction scores pairs of final node states, and graph classification summarizes each graph with SheafConvPool.

3.3 Task-Specific Input Preprocessing

All three tasks start from the same representation. The heterogeneous graph is flattened into one node table and one directed edge table that retain each node's type and each edge's relation type, and raw features are zero-padded to a common width, written x_v. What differs across tasks is the supervision available to describe how relations behave; in every case it is derived from the training split alone, and validation and test labels or links never enter this stage.

Node classification. A single label-propagation pass, clamped to the training labels, gives every node a soft class distribution $\widehat{y}_v$. For each relation type we then record four endpoint-agreement scores: between propagated class distributions, between structural summaries such as degree and typed-path counts, between feature-derived summaries, and as normalized mutual information between the endpoint distributions. Stacked over all relations, these form one global vector $m \in \mathbb{R}^{4|\mathcal{R}|}$ that is appended to every node,

$$x_v = x_v^{\mathrm{raw}} \,\|\, m, \tag{30}$$

so the encoder can see which relations tend to link disagreeing endpoints instead of having to infer it from features alone.

Link prediction. No node labels exist to propagate, so the encoder receives x_v unchanged. Instead, AMPSS uses a two-channel signal recording how often a node appears as the source or the target of a training-positive edge, and consults it only when scoring candidate link schemas.

Graph classification. The diagnostic is computed per fold rather than per node. Within the training graphs of the current fold, AMPSS retains typed path schemas whose presence differs across classes, so the schema pool reflects only patterns that separate the training labels.

3.4 Node Stalks and Sheaf Restrictions Initialization

This stage instantiates the first-coboundary machinery for the flattened heterogeneous graph. The type-specific encoder produces initial node stalks, and relation-conditioned restriction maps define how each edge compares its endpoints. This is inherited sheaf structure rather than a separate meta-path contribution; it supplies the δ_0 operator that the later edge-to-face construction must compose with.

Type-specific node states. Different node types need not share a feature vocabulary or input dimension. We therefore map each type into the common stalk space before comparing adjacent nodes:

$$h_v^{(0)} = \mathrm{Dropout}\big[\mathrm{GELU}\big(\mathrm{LN}(W_{\phi(v)} x_v + b_{\phi(v)}) + q_{\phi(v)}\big)\big] \in \mathbb{R}^d, \tag{31}$$

where W_t, b_t are type-specific parameters and q_t is a learned type embedding. A learned node table supplies $h_v^{(0)}$ when raw attributes are unavailable. We initialize the diffused node cochain as $x^{(0)} = h^{(0)}$, and represent each relation r by $z_r \in \mathbb{R}^{d_r}$.

Relation-conditioned restrictions. Given an oriented edge $e = (u \xrightarrow{r} v)$, two relation-conditioned networks infer $F_{u\trianglelefteq e}, F_{v\trianglelefteq e} \in \mathbb{R}^{d\times d}$ from $h_u^{(0)}$, $h_v^{(0)}$, and z_r. Each restriction is scaled orthogonal:

$$F_{v\trianglelefteq e} = \lambda_{v,e} Q_{v,e}, \qquad Q_{v,e}^{\mathsf{T}} Q_{v,e} = I, \qquad \lambda_{v,e} > 0, \tag{32}$$

and analogously for $F_{u\trianglelefteq e}$. The orthogonal factor aligns local coordinates, while the positive scale adjusts the strength of the incidence.

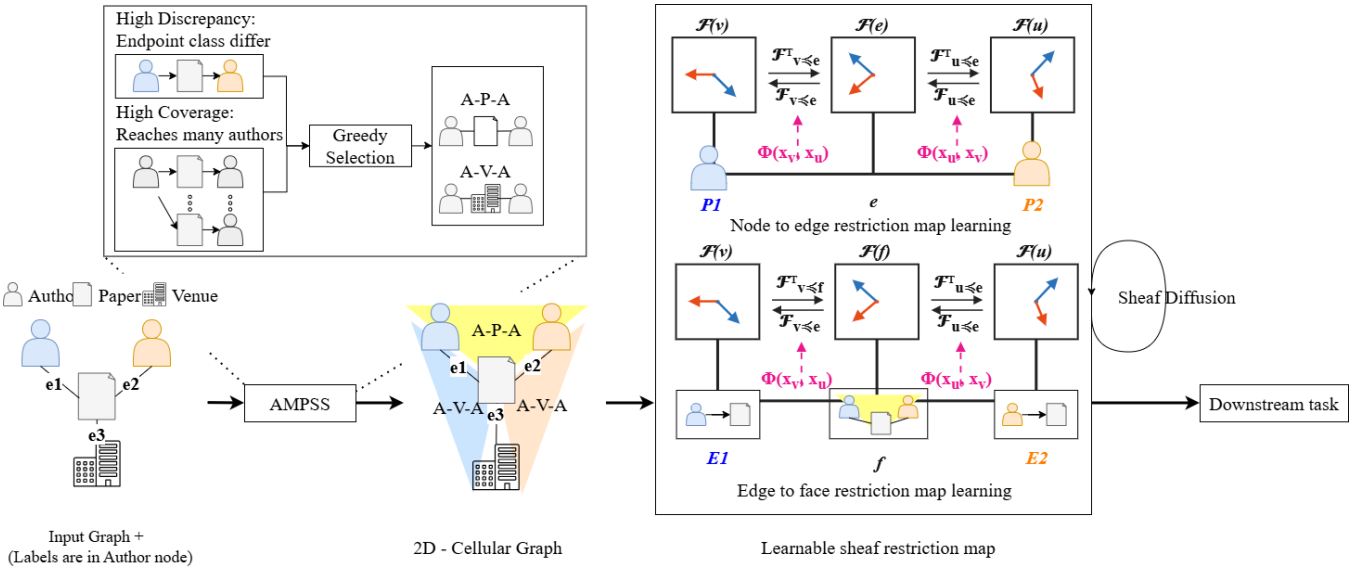


Figure 3: Overview of H2S-NN. Typed graph data enter a training-only AMPSS selection stage; selected schemas and weighted path instances then parameterize the node-edge-face geometry. Node and edge cochains diffuse in parallel before task-specific node, link, or graph prediction, with graph classification routed through SheafConvPool.

These restrictions define the first coboundary immediately:

$$(\delta_0 x)_e = F_{v \trianglelefteq e} x_v - F_{u \trianglelefteq e} x_u. \tag{33}$$

Thus, $\delta_0 x$ measures endpoint disagreement after relation-specific transport rather than raw feature difference.

Independent edge initialization. Endpoint disagreement is informative, but it need not span every useful edge signal. We initialize the edge cochain with a separate network,

$$s_e^{(0)} = \mathrm{MLP}_{\mathrm{init}}\left(h_u^{(0)} \,\|\, h_v^{(0)} \,\|\, z_r \,\|\, (\delta_0 h^{(0)})_e\right), \tag{34}$$

so $s^{(0)} \neq \delta_0 h^{(0)}$ in general. This preserves an edge component outside $\mathrm{im}\,\delta_0$ for the subsequent lower- and upper-Hodge interactions.

3.5 AMPSS: Adaptive Meta-Path Schema-to-Cell Selection

AMPSS separates economical structure search from task-driven refinement. The first stage is a non-differentiable, training-only construction of a candidate pool; the second stage learns soft weights for the retained schemas and their concrete instances jointly with H2S-NN.

Stage I: candidate generation and preselection. The first stage constructs a small set of typed higher-order supports before the neural model is trained. It does so at the level of the heterogeneous schema rather than by immediately traversing all concrete node paths. We first form a directed type graph whose nodes are entity types and whose directed edges are relation types, e.g., $P \xrightarrow{\text{writes}^{-1}} A$ and $A \xrightarrow{\text{writes}} P$ in a paper–author network. A bounded depth-first search over this type graph enumerates valid schema candidates $\sigma = (t_0, r_1, t_1, \ldots, r_\ell, t_\ell)$ subject to the allowed length range and, when required by the task, the start and end target types. This step therefore proposes only type-valid relational routes; it does not yet decide that any route is useful.

For each candidate schema, AMPSS then samples concrete instances in the original graph. Starting from nodes of type t_0, it follows the prescribed relation sequence $(r_1, \ldots, r_\ell)$ and records paths $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$ whose node and edge types match the schema. Separate budgets cap the number of candidate schemas, the number of instances expanded from one anchor node, the number of instances retained for one schema, and the total number of materialized instances. These caps make the construction economical while still exposing the model to the main typed relational patterns present in the graph.

The preselection score is designed to keep schemas that are both informative and usable as face-like supports. For node classification, training labels are propagated for a fixed number of steps to obtain soft endpoint distributions $\widehat{y}_v$. A concrete path receives a large discrepancy when its two endpoints carry different training-derived class evidence, and a schema receives the average discrepancy of its sampled instances:

$$d(f) = \left\|\widehat{y}_{v_0} - \widehat{y}_{v_\ell}\right\|_2^2, \qquad D(\sigma) = \frac{1}{|\mathcal{I}_\sigma|} \sum_{f \in \mathcal{I}_\sigma} d(f). \tag{35}$$

Intuitively, this favors schemas whose endpoints expose nontrivial class contrast under the training signal, which can create useful edge-level disagreement for the subsequent Hodge diffusion. For link prediction, the same idea is implemented with two propagated endpoint-role channels that mark source and target positions using training-positive edges only. Neither validation nor test labels or links contribute to these statistics.

For graph classification, endpoint discrepancy is not the relevant signal. Let $n_{\sigma,c}^{\mathrm{pres}}$ be the number of training graphs of class c in which schema σ occurs, N_c the number of training graphs in that class,

and n_σ the total number of its materialized instances. The presence score is

$$P_G(\sigma) = \mathrm{Var}_c\left(\frac{n^{\mathrm{pres}}_{\sigma,c}}{\max(N_c, 1)}\right) + 10^{-3}\log(1 + n_\sigma). \tag{36}$$

This favors schemas whose presence differs across training classes, with frequency used only as a small tie breaker.

For node and link tasks, AMPSS greedily builds the selected pool one schema at a time. The score combines four quantities. The discrepancy $D(\sigma)$ measures task relevance. The endpoint coverage $C(\sigma)$ measures how many target-type endpoints are reached by the sampled instances, so a schema that appears only in a tiny corner of the graph is not selected merely because it has a high local score. The type diversity $V(\sigma)$ rewards schemas that pass through multiple semantic types rather than repeatedly using the same type pattern. Finally, $J(\sigma, \sigma')$ measures endpoint-set overlap with an already selected schema σ', discouraging several selected schemas from describing nearly the same region of the graph. Given the current pool S, the greedy score is

$$q(\sigma \mid S) = \frac{D(\sigma)^{a_d} C(\sigma)^{a_c} V(\sigma)^{a_v}}{1 + \max_{\sigma' \in S} J(\sigma, \sigma')}, \tag{37}$$

The numerator prefers schemas that are informative, sufficiently covered, and semantically varied; the denominator lowers the priority of schemas that are redundant with the pool already chosen. Graph tasks use the class-presence score $P_G(\sigma)$ instead of Equation **??**. After this greedy pass, the selected schemas are materialized as ordered path supports, together with the closure pairs used by the node-side correction. This entire stage fixes the candidate support and is intentionally non-differentiable; the next stage decides how strongly the retained schemas and their individual instances should participate in the learned operator.

Stage II: differentiable schema and instance refinement. Preselection decides what can be considered; it does not force every retained structure to contribute equally. We learn one gate per schema and a state-dependent gate for every path instance:

$$\begin{aligned} \alpha_\sigma &= \mathrm{sigmoid}(\theta_\sigma), \\ \beta_f &= \mathrm{sigmoid}\Big(g_\beta(h^{(0)}_{v_0} \parallel h^{(0)}_{v_\ell} \parallel e_\sigma)\Big), \\ w_f &= \alpha_{\sigma(f)}\beta_f. \end{aligned} \tag{38}$$

Here e_σ is a learned schema embedding. Schema logits are warm-started from standardized preselection scores, while β_f lets two instances of the same schema have different effects. During training, partial-schema propagation retains each schema with probability p_{keep} and rescales a retained gate by $1/p_{\mathrm{keep}}$, following the stabilization idea of partial message propagation [**?**].

The discovery loss applies complementary pressures rather than a single hard selection rule:

$$\begin{aligned} \mathcal{L}_{\mathrm{disc}} = \lambda_s \mathcal{L}_{\text{sparse-schema}} + \lambda_i \mathcal{L}_{\text{sparse-instance}} + \lambda_r \mathcal{L}_{\mathrm{redundancy}} \\ + \lambda_v \mathcal{L}_{\mathrm{coverage}} + \lambda_{\mathrm{cost}} \mathcal{L}_{\mathrm{cost}} + \lambda_H \mathcal{L}_{\mathrm{entropy}}. \end{aligned} \tag{39}$$

Schema and instance terms discourage diffuse activation; redundancy penalizes similar active coverage profiles; the negative coverage reward discourages a trivial empty pool; cost suppresses long, frequently instantiated schemas; and entropy minimization sharpens schema decisions. These terms refine only the preselected pool and do not make candidate generation differentiable.

Algorithm 1 AMPSS candidate preselection and differentiable refinement

Require: Typed relation graph, training split, schema and instance budgets

Stage I: Candidate generation and training-only preselection
Enumerate bounded typed schemas and sample concrete instances
Compute node/link discrepancy scores or graph presence scores
Greedily retain a compact pool Σ and materialize its path supports
Initialize θ_σ from standardized preselection scores
for each training epoch **do**
  Stage II: Differentiable schema and instance refinement
  Use uniform face weights during warm-up
  Compute α_σ, β_f, and $w_f = \alpha_{\sigma(f)}\beta_f$
  Apply partial-schema propagation with keep probability p_{keep}
  Run matrix-free sheaf-Hodge diffusion and the task readout
  Optimize Equation **??**
end for

3.6 2D-cellular Sheaf-Hodge Geometry Construction

A cellular complex represents a structure using cells of different dimensions. General graphs are 1-D cellular complexes where Nodes are 0-cells, edges are 1-cells. Unlike another sheaf diffusion network that works on 1-D cellular complexes graph [**? ? ? ?**], however, for our H2S-NN we construct graphs as 2-D cellular complexes by adding faces produces a two-dimensional complex. We write $\sigma \trianglelefteq \tau$ when a lower-dimensional cell σ is incident to a higher-dimensional cell τ. A cellular sheaf $\mathcal{F}$ equips each cell σ with a local vector space, or *stalk*, $\mathcal{F}(\sigma)$, and assigns a linear restriction map $F_{\sigma \trianglelefteq \tau} : \mathcal{F}(\sigma) \to \mathcal{F}(\tau)$ to every incident pair. These maps transport information from different cells into common spaces and must compose consistently along incidence chains.

On a graph $G = (V, E)$, the relevant stalks live on nodes and edges. For each incident pair $v \trianglelefteq e$, the restriction map $F_{v \trianglelefteq e} : \mathcal{F}(v) \to \mathcal{F}(e)$ transports the node representation into the shared edge space. Node and edge assignments form the cochain spaces

$$C^0(G, \mathcal{F}) = \bigoplus_{v \in V} \mathcal{F}(v), \qquad C^1(G, \mathcal{F}) = \bigoplus_{e \in E} \mathcal{F}(e). \tag{40}$$

Given an oriented edge $e = (u \to v)$, the first coboundary operator $\delta_0 : C^0(G, \mathcal{F}) \to C^1(G, \mathcal{F})$ measures disagreement between the transported endpoint representations:

$$(\delta_0 x)_e = F_{v \trianglelefteq e} x_v - F_{u \trianglelefteq e} x_u. \tag{41}$$

The corresponding sheaf Laplacian is $L_0 = \delta_0^{\mathsf{T}} \delta_0$, with energy $x^{\mathsf{T}} L_0 x = \|\delta_0 x\|_2^2$. When all stalks are one-dimensional and all restriction maps are identities, L_0 reduces to the ordinary graph Laplacian.

3.7 Path-Supported Edge-Face Geometry

Consider a selected $P-A-P$ instance $f = (v_0, e_1, v_1, e_2, v_2)$. The model retains the ordered path edges $\partial_{\text{path}} f = (e_1, e_2)$ as the path part of the boundary: it does not replace the instance by a shortcut edge between its endpoint papers. For a general instance $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$, an edge token combines its endpoint states and relation,

$$\begin{aligned} g_i &= \mathrm{MLP}_g\Big(h^{(0)}_{v_{i-1}} \parallel h^{(0)}_{v_i} \parallel z_{\psi(e_i)}\Big), \\ (o_1, \ldots, o_\ell) &= \mathrm{LSTM}(g_1, \ldots, g_\ell). \end{aligned} \tag{42}$$

The ordered encoder distinguishes positions along the path. Attention then pools the sequence into a single summary, which produces one free map per instance:

$$\pi_{fi} = \mathrm{softmax}_i\left(\frac{o_i^\top a}{\sqrt{d_h}}\right), \qquad A_f = \mathrm{reshape}\Big(W_F \textstyle\sum_i \pi_{fi} o_i + b_F,\ d, d\Big). \tag{43}$$

Attention shapes A_f; it deliberately does not rescale individual edges' maps, because a non-uniform per-edge factor would destroy the boundary cancellation established below.

The per-edge maps are then *not* emitted independently. A cellular sheaf is a functor on the face poset, so an edge-to-face map must compose with the node-to-edge maps beneath it; emitting both from separate networks breaks that functoriality, and with it the cochain property. We therefore carry a transport to the base vertex v_0 along the boundary:

$$\begin{aligned} T_{v_0} &= A_f, \qquad T_{v_i} = T_{v_{i-1}} F^{-1}_{u \trianglelefteq e_i} F_{v \trianglelefteq e_i}, \\ F_{e_i \trianglelefteq f} &= T_{v_{i-1}} F^{-1}_{u \trianglelefteq e_i}, \end{aligned} \tag{44}$$

where $u = v_{i-1}$ and $v = v_i$ are the endpoints of e_i. The inverse is available in closed form: by Eq. (??) each map is a positive scale times an orthogonal factor, so $F^{-1} = F^\top/\lambda^2$, and no linear solve is required. Each map $F_{e_i \trianglelefteq f} : \mathcal{F}(e_i) \to \mathcal{F}(f)$ still transports the i-th edge state into the shared face stalk, and still varies per instance and per relation; what changes is that a single $d \times d$ matrix per instance is learned freely rather than ℓ of them.

Let $\eta(e, f) \in \{-1, +1\}$ be the stored incidence orientation. The face gate enters the weighted second coboundary:

$$(\delta_{1,w} s)_f = \sqrt{w_f} \sum_{e \in \partial f} \eta(e, f) F_{e \trianglelefteq f} s_e. \tag{45}$$

Writing $B_f s = \sum_{e \in \partial f} \eta(e, f) F_{e \trianglelefteq f} s_e$, its upper-Hodge contribution is therefore

$$\delta^\top_{1,w} \delta_{1,w} = \sum_{\sigma \in \Sigma} \sum_{f \in \mathcal{I}_\sigma} w_f B_f^\top B_f. \tag{46}$$

The square root is important: it makes each face contribute linearly in w_f, not quadratically.

An open path is not the boundary of a 2-cell: a cell is attached along a cycle. Each instance therefore also carries a synthetic closure 1-cell $e_{\text{cl}} = (v_\ell \to v_0)$, oriented along the cycle, which completes $\partial f = (e_1, \ldots, e_\ell, e_{\text{cl}})$. It is a stateful edge with its own stalk, and its restriction maps are *defined* rather than learned:

$$F_{v_0 \trianglelefteq e_{\text{cl}}} = I, \qquad F_{v_\ell \trianglelefteq e_{\text{cl}}} = T^{-1}_{v_0} T_{v_\ell}, \qquad F_{e_{\text{cl}} \trianglelefteq f} = A_f. \tag{47}$$

The middle map reduces to $\prod_i F^{-1}_{u \trianglelefteq e_i} F_{v \trianglelefteq e_i}$, the holonomy of the path itself: it is independent of A_f and adds no parameters. The closure cell is thus a bookkeeping device that makes "traverse this path and return" well posed, not an asserted relation between the endpoints. Figure ?? makes this boundary explicit.



Figure 4: AMPSS-selected path instances define a sheaf-Hodge operator. Currently in construction.

Proposition 1 (Exact cochain condition). *For every instance whose node sequence is a simple path, the maps of Eqs.* (??)–(??) *satisfy* $\delta_{1,w}\delta_0 = 0$ *identically, for every node cochain and every value of the learned parameters, with no assumption that the sheaf has trivial holonomy.*

Proof. By construction $F_{e_i \trianglelefteq f} F_{u \trianglelefteq e_i} = T_{v_{i-1}}$ and $F_{e_i \trianglelefteq f} F_{v \trianglelefteq e_i} = T_{v_i}$, so the i-th boundary term of $\delta_{1,w}\delta_0 x$ equals $T_{v_i} x_{v_i} - T_{v_{i-1}} x_{v_{i-1}}$, and summing over the path telescopes to $T_{v_\ell} x_{v_\ell} - T_{v_0} x_{v_0}$. Eq. (??) gives $F_{e_{\text{cl}} \trianglelefteq f} F_{v_0 \trianglelefteq e_{\text{cl}}} = T_{v_0}$ and $F_{e_{\text{cl}} \trianglelefteq f} F_{v_\ell \trianglelefteq e_{\text{cl}}} = T_{v_\ell}$, so the closure term contributes $T_{v_0} x_{v_0} - T_{v_\ell} x_{v_\ell}$ and the total vanishes. Simplicity is required because the transport assigns one map per node position; a revisited node — including a coincident endpoint pair — would be assigned two values. □

Empirically the residual $\|\delta_{1,w}\delta_0 h^{(0)}\| / \|\delta_0 h^{(0)}\|$ is at the level of single-precision roundoff ($\sim 10^{-7}$) on all evaluated datasets, for both manual and AMPSS schema sets.

3.8 Matrix-Free Sheaf-Hodge Diffusion

The two coboundaries induce one node operator and one edge operator:

$$L_0 x = \delta_0^\top \delta_0 x, \tag{48}$$

$$L_1 s = \underbrace{\delta_0 \delta_0^\top s}_{\text{lower edge term}} + \underbrace{\delta^\top_{1,w} \delta_{1,w} s}_{\text{upper edge term}}. \tag{49}$$

The lower edge term compares edge signals through their incident nodes; the upper term couples the edges of a boundary cycle through their shared face. Because the closure 1-cell of Section ?? is an ordinary member of the edge set, it enters δ_0 — and hence L_0 and the lower edge term — exactly like a data edge; no separate node-side correction is applied. Its restriction maps are the path holonomy of Eq. (??), so it introduces no parameters of its own, and a scale $\gamma > 0$ on them is absorbed by the compensating factor in $F_{e_{\text{cl}} \trianglelefteq f}$, leaving Proposition ?? unaffected while tuning how strongly the boundary cycle closes at the node level.

Starting from $x^{(0)} = h^{(0)}$ and the independent $s^{(0)}$, K explicit Euler steps give

$$\begin{aligned} x^{(k+1)} &= x^{(k)} - \eta_0 L_0 x^{(k)}, \\ s^{(k+1)} &= s^{(k)} - \eta_1 L_1 s^{(k)}, \qquad k = 0, \dots, K-1. \end{aligned} \tag{50}$$

The positive step sizes may be learned. Node and edge cochains evolve in parallel under geometry inferred from the same initial node states; there is no cross-rank recurrence between Euler steps. Layer normalization yields the terminal states $x^\star$ and $s^\star$.

Execution is matrix-free: the model applies restrictions in batches and aggregates incident contributions, without materializing global $Nd \times Nd$ or $Md \times Md$ matrices. Three implementation boundaries remain consequential. First, $\delta_{1,w}$ spans the complete boundary cycle: the ℓ path edges together with the closure 1-cell. Second, signed incidences are supported, and the uniform +1 used here is the correct boundary map precisely because enumeration traverses every edge along its stored direction and stores the closure cell oriented along the cycle; the pipeline asserts this rather than assuming it, so a future change to reverse traversal fails loudly instead of silently invalidating the signs. Third, because faces must be simple paths for the transport of Eq. (??) to be well defined, instances that revisit a node — including those whose endpoints coincide — are excluded during enumeration. On the manual meta-path templates this removes a negligible fraction of instances, since the search backfills its per-anchor budget with simple paths. Unlike earlier path-supported sheaf constructions, $\delta_{1,w}\delta_0 = 0$ is not encouraged by a penalty but holds identically (Proposition ??).

3.9 Downstream Readouts

Node classification. Let $\bar{s}_v^\star = |E(v)|^{-1} \sum_{e \ni v} s_e^\star$, with a zero vector for an isolated node. The target-node classifier retains the terminal node state, the pre-diffusion state, and the local edge summary:

$$\widehat{y}_v = \mathrm{MLP}_{\mathrm{node}}\left(x_v^\star \,\|\, h_v^{(0)} \,\|\, \bar{s}_v^\star\right). \tag{51}$$

The third view gives the supervised node head direct access to information transformed by the upper edge operator.

Link prediction. Candidate endpoints are decoded from their final node states using a dot product, a relation-conditioned DistMult decoder, or a concatenation MLP. The current link head does not directly consume $s^\star$; consequently, its task loss reaches the upper-edge pathway only through shared training terms, not through an edge-state link decoder.

Graph classification. A graph head must be invariant under node and edge permutations and orthogonal stalk gauges. SheafConvPool meets these requirements while retaining both terminal and initial node views and the learned edge cochain. It is a single-level terminal readout.

3.10 SheafConvPool

For each node v, SheafConvPool stacks three aligned views,

$$Z_v^{(0)} = \left[x_v^\star \quad h_v^{(0)} \quad \bar{s}_v^\star\right] \in \mathbb{R}^{d \times 3}. \tag{52}$$

For each stored edge e, it similarly forms

$$Z_e^{(1)} = \left[(\delta_0 x^\star)_e \quad (\delta_0 h^{(0)})_e \quad s_e^\star\right] \in \mathbb{R}^{d \times 3}. \tag{53}$$



Figure 5: SheafConvPool graph readout. Node and edge view stacks become orthogonal-gauge invariant through $Z^\top Z$. Separate projections and invariant attention produce weighted means and covariances; their node/edge summaries are concatenated for graph classification. The actual picture is currently in construction.

The corresponding 3×3 Gram descriptors are $G_i^{(r)} = (Z_i^{(r)})^\top Z_i^{(r)}$, for rank $r \in \{0, 1\}$. Under an orthogonal local gauge $Z_i^{(r)} \mapsto Q_i Z_i^{(r)}$, $G_i^{(r)}$ is unchanged. The readout therefore learns only from coordinate-independent inner products among its three views.

Separate node and edge networks project the flattened descriptors, $u_i^{(r)} = \rho_r(\mathrm{vec}\, G_i^{(r)}) \in \mathbb{R}^{d_p}$, and score them with invariant attention. Within each graph and rank,

$$a_i^{(r)} = \frac{\exp(g_r(u_i^{(r)}))}{\sum_{j \in C_r} \exp(g_r(u_j^{(r)}))}, \qquad \mu_r = \sum_{i \in C_r} a_i^{(r)} u_i^{(r)}, \tag{54}$$

where C_0 and C_1 are the graph's nodes and edges. A mean alone cannot record whether informative descriptors are concentrated or dispersed, so the readout also computes

$$\Sigma_r = \sum_{i \in C_r} a_i^{(r)} (u_i^{(r)} - \mu_r)(u_i^{(r)} - \mu_r)^\top. \tag{55}$$

The graph representation concatenates μ_0, the upper triangle of Σ_0, μ_1, and the upper triangle of Σ_1; an MLP maps this vector to graph logits. Softmax normalization and moment summation make the construction permutation invariant, while the Gram descriptors make it invariant under the model's orthogonal stalk gauges.

3.11 Training Objective

The geometry carries no auxiliary loss. The task term is cross-entropy for single-label node and graph classification, binary cross-entropy for multi-label classification and link prediction, with the appropriate training mask or sampled training pairs, and the only additional term is the AMPSS schema-selection objective:

$$\mathcal{L} = \mathcal{L}_{\mathrm{task}} + \rho(t)\mathcal{L}_{\mathrm{disc}}. \tag{56}$$

During discovery warm-up, gates are bypassed by uniform face weights and $\rho(t) = 0$. After warm-up, the schema and instance gates activate; $\rho(t)$ is either applied at full strength or annealed linearly to one over a configured interval. This schedule lets task gradients establish a useful path geometry before sparsity, redundancy, and

cost pressures dominate. For manual schema sets $\mathcal{L}_{\text{disc}}$ is absent and the objective reduces to the task loss alone.

That the geometric terms are absent is a consequence of the construction, not an omission. A chain penalty is unnecessary because Proposition ?? makes $\delta_{1,w}\delta_0$ vanish identically; on an open path such a penalty is in any case unsatisfiable except by degenerate face maps, since the two endpoint blocks of $\delta_{1,w}\delta_0$ each appear in a single term while the node-to-edge maps are invertible. A consistency penalty $\|s^\star - \delta_0 x^\star\|^2$ is counterproductive: x evolves under L_0 alone and the edge channel never feeds back into it, so driving $s^\star$ to $\delta_0 x^\star$ would make the edge cochain a deterministic image of the node cochain and the faces would contribute nothing. A face-level Dirichlet penalty $\|\delta_{1,w} s^\star\|^2$ is counterproductive for the same reason at one rank higher: with the cochain condition exact, $\delta_{1,w} s^\star$ is precisely the curl component of the edge cochain and the only part the upper Hodge term $\delta_{1,w}^{\mathsf{T}}\delta_{1,w}$ acts on, so shrinking it drives the face branch inert. The edge cochain is therefore left free at both ranks.

4 Experiments

We evaluated H2S-NN with AMPSS on a range of heterogeneous graph tasks, including node classification, link prediction, and graph classification. On the graph classification task, we also evaluated the single-level SheafConvPool readout.

In general, the model selection was performed using hyperparameter sweeps. The full per-dataset hyperparameters are listed in Appendix ??. The code is available at placeholder_for_code_repository.com.

In Tables ??, ??, and ??, we group the baselines introduced in each subsection's *Baselines* paragraph into four categories: Hom./hom., Het./hom., Hom./het., and Het./het. The first term indicates whether the model assumes a homogeneous or heterogeneous graph, while the second indicates whether it is designed for homophilic or heterophilic settings. H2S-NN is reported separately under Ours. Since HetSheaf does not provide a public implementation, we report the results published in its original paper, which follows the same experimental settings as ours. To ensure a fair comparison, we conducted the experiments with 5 different random seeds and reported the average performance with standard deviation.

4.1 Node Classification

Datasets. On the node classification task, we used three benchmark heterogeneous graph datasets: DBLP [?] and IMDB [?], and ACM [?]. The statistics of these datasets are summarized in Table ??.

Table 1: Node-classification datasets and their semantic meta-paths. The N column indicates nodes, E indicates the number of edges, and C indicates classes.

Dataset	#N	#N types	#E	#E types	Classes	Target	Meta-paths
DBLP	26,128	4	239,566	6	4	author	APA, APCPA, APTPA
IMDB	21,420	4	86,642	6	5	movie	MAM, MDM
ACM	10,942	4	547,806	7	3	paper	PAP, PSP

Baselines. We compared H2S-NN against four groups of baselines: (i) homogeneous homophilic models, including GCN [?], GAT [?], and ChebNetII [?]; (ii) homogeneous heterophilic models, including NSD [?] and CSNN [?]; (iii) heterogeneous homophilic models, including HAN [?], SeHGNN [?], HiNormer [?], and PSHGCN [?]; and heterogeneous heterophilic models, including HDHGR [?], Hetero^{2}Net [?], H2SGNN [?], HetSheaf [?], and LatGRL [?].

Evaluation Settings. For the node classification tasks experimental settings, we followed the HGB benchmark [?] by using the training/validation/test splits in a ratio of 20%/20%/60% on transductive learning. We used Macro-F1 and Micro-F1 as the evaluation metrics. For our H2S-NN, we conducted the experiment by including AMPSS.

Results. From the results in Table ??, H2S-NN achieved the best performance on all three benchmark datasets in node classification tasks. H2S-NN outperforms every method that addresses heterogeneity without heterophily (Het./hom.) or heterophily on homogeneous graphs (Hom./het.), consistent with treating type-aware restriction and heterophily-robust comparison as complementary rather than substitutable. Against the strongest heterogeneous heterophilic (Het./het.) baselines, which already combine both properties, the remaining margin is consistent but smaller, which we attribute to H2S-NN additionally retaining concrete meta-path instances as operator-level structure rather than only adjusting node-level message passing.

4.2 Graph classification

Datasets. On the graph classification task, we used four benchmark heterogeneous graph datasets from TUDataset [?]: biological graphs, including PROTEINS and ENZYMES, and chemical compound graphs, including MUTAG and NCI1. To make those graphs heterogeneous, we treated the node and edge labels supplied by TUDataset as different node and edge types. Table ?? gives the statistics of the four TUDataset graph-classification benchmarks.

Table 3: Graph-classification dataset statistics. The N column indicates nodes, E indicates the number of edges, and C indicates classes.

Dataset	Graphs	#N types	#E	#E types	Target
MUTAG	188	7	7,442	4	mutagenicity
PROTEINS	1,113	3	162,088	1	enzyme/non-enzyme
ENZYMES	600	3	19,580	1	enzyme class
NCI1	4,110	37	49,205	1	anti-cancer activity

Baselines. We compared H2S-NN against four groups of baselines: (i) homogeneous homophilic models, including GCN [?] and GAT [?]; (ii) homogeneous heterophilic models, including NSD [?] and CSNN [?]; (iii) heterogeneous homophilic models, including HAN [?] and PSHGCN [?]; and (iv) heterogeneous heterophilic models, including HDHGR [?], Hetero^{2}Net [?], H2SGNN [?], HetSheaf [?], and LatGRL [?].

Evaluation Settings. Following the graph classification research [? ?], we used 10-fold cross-validation and reported the best evaluation score obtained during training. We used Macro-F1 and Micro-F1 as the evaluation metrics. For our H2S-NN, we conducted the experiment by including AMPSS and SheafConvPool. To ensure a fair

Table 2: Performance on node classification benchmarks reporting in Macro-F1 and Micro-F1. Best values are bold.

		DBLP (59.76)		ACM (72.39)		IMDB (50.04)	
Group	**Model**	Macro	Micro	Macro	Micro	Macro	Micro
Hom./hom.	GCN	90.77 ± 0.76	90.28 ± 0.20	91.63 ± 0.44	91.38 ± 0.39	62.33 ± 0.58	67.69 ± 0.59
	GAT	94.39 ± 1.03	92.52 ± 0.27	92.07 ± 0.57	90.71 ± 0.21	62.13 ± 0.26	68.09 ± 0.48
	ChebNetII	92.89 ± 1.07	92.67 ± 0.50	91.70 ± 0.27	91.59 ± 0.27	62.97 ± 0.66	68.47 ± 0.55
Het./hom.	NSD	92.96 ± 0.21	93.42 ± 0.37	92.33 ± 0.78	92.05 ± 0.69	61.61 ± 0.26	65.23 ± 0.51
	CSNN	93.73 ± 0.95	92.12 ± 0.19	93.13 ± 0.94	92.81 ± 0.83	62.92 ± 0.34	62.59 ± 0.22
Hom./het.	HAN	91.78 ± 0.92	92.24 ± 0.86	90.88 ± 0.67	91.61 ± 1.07	61.95 ± 0.36	67.76 ± 0.51
	SeHGNN	95.52 ± 0.98	95.48 ± 0.78	94.38 ± 1.00	94.04 ± 0.73	72.01 ± 0.65	72.59 ± 0.18
	HiNormer	94.06 ± 0.40	94.14 ± 0.46	91.87 ± 0.26	89.64 ± 0.39	64.13 ± 0.19	67.53 ± 0.32
	PSHGCN	95.58 ± 0.80	95.70 ± 0.54	93.66 ± 0.53	94.70 ± 0.39	71.82 ± 0.34	74.34 ± 0.84
Het./het.	HDHGR	95.35 ± 0.81	95.00 ± 0.77	93.99 ± 0.35	94.13 ± 0.87	59.41 ± 0.22	59.63 ± 0.35
	Hetero2Net	93.54 ± 1.11	93.54 ± 0.79	94.05 ± 0.71	92.87 ± 0.82	65.34 ± 0.68	70.63 ± 0.69
	H2SGNN	95.82 ± 0.41	95.11 ± 0.22	94.90 ± 0.49	94.08 ± 0.44	74.23 ± 0.31	75.29 ± 0.86
	HetSheaf	93.87 ± 0.00	93.77 ± 0.00	95.09 ± 0.00	94.94 ± 0.00	64.65 ± 0.00	67.18 ± 0.00
	LatGRL	94.32 ± 0.44	94.51 ± 0.42	93.47 ± 0.71	92.29 ± 0.43	58.27 ± 0.46	58.81 ± 0.65
Ours	**H2S-NN**	**96.01 ± 0.58**	**96.31 ± 0.40**	**95.91 ± 1.15**	**95.72 ± 0.68**	**75.05 ± 0.22**	**76.68 ± 0.19**

comparison, we conducted the experiments with 5 different random seeds, each with 100 epochs for early stopping, and reported the average performance with standard deviation. For the ENZYMES dataset, we used a one-vs-rest (OVR) strategy.

Results. From the results in Table **??**, H2S-NN achieved the best performance on all four benchmark datasets in graph classification tasks, tying the MUTAG ceiling and leading PROTEINS, ENZYMES, and NCI1 outright over every compared heterogeneous heterophilic baseline. The margin is largest on PROTEINS (+3.88 Macro-F1 over the closest baseline, HetSheaf), which suggests that treating the typed node and edge labels as heterogeneous structure, rather than as plain homogeneous graphs with categorical attributes, remains useful even outside the citation- and social-network domains where heterogeneity is usually studied.

4.3 Heterogeneous link prediction

Datasets. On the link prediction task, we used four benchmark heterogeneous graph datasets from HGB [**?**]: user-item interaction networks, including LastFM, YouTube, and Amazon; and a biological knowledge graph, including PubMed. The statistics of these datasets are summarized in Table **??**.

Table 5: Heterogeneous link-prediction dataset statistics. The N column indicates nodes, E indicates the number of edges, and C indicates classes.

Dataset	#N	#N types	#E	#E types	Target
LastFM	20,612	3	141,521	3	user-artist
PubMed	63,109	4	244,986	10	disease-disease
YouTube	2,000	1	1,177,130	5	user-user
Amazon	10,099	1	148,659	2	product-product

Baselines. We compared H2S-NN against four groups of baselines: (i) homogeneous homophilic models, including GCN [**?**] and GAT [**?**]; (ii) homogeneous heterophilic models, including NSD [**?**] and CSNN [**?**]; (iii) heterogeneous homophilic models, including HAN [**?**] and PSHGCN [**?**]; and (iv) heterogeneous heterophilic models, including HDHGR [**?**], Hetero^{2}Net [**?**], H2SGNN [**?**], HetSheaf [**?**], and LatGRL [**?**].

Evaluation Settings. For the link prediction tasks experimental settings, we followed the HGB benchmark [**?**] by using the training/validation/test splits in a ratio of 20%/20%/60% on transductive learning. We used AUPR and AUROC as the evaluation metrics. For our H2S-NN, we conducted the experiment by including AMPSS. For the negative sampling strategy, we followed the approach in the HGB benchmark dataset [**?**] by sampling negative edges for each positive edge in a ratio of 1:1 from the 2-hop neighborhood.

Results. From the results in Table **??**, H2S-NN achieves the highest mean score on all four link-prediction datasets and all eight reported metrics. The improvement is consistent across both multi-node-type graphs, LastFM and PubMed, and single-node-type multi-relation graphs, Amazon and YouTube. The margins are not uniform: PubMed AUPR is close to PSHGCN, while the separation is larger on PubMed AUROC and especially on Amazon. This pattern suggests that the learned typed sheaf operators and AMPSS path-instance weighting can support link prediction even when heterogeneity is expressed mainly through relation types rather than through multiple node types.

5 Discussion

5.1 Ablation Study: Contribution of AMPSS

To evaluate the effectiveness of AMPSS, we compared the full H2S-NN on the DBLP dataset on node classification tasks with variants using no meta-path cells, predefined meta-paths, AMPSS without the instance-level gate, and the full AMPSS. As shown in Table **??**, removing meta-path cells resulted in a clear performance drop, decreasing Macro-F1 and Micro-F1 from 96.01/96.31 to 93.85/93.72. This confirms that incorporating higher-order semantic relations is important for node classification. Using predefined meta-paths recovered most of the lost performance, while replacing them with

Table 4: Performance on graph classification benchmarks reporting in Macro-F1 and Micro-F1. Best values are bold.

		MUTAG		PROTEINS		ENZYMES		NCI1	
Group	**Model**	Ma.	Mi.	Ma.	Mi.	Ma.	Mi.	Ma.	Mi.
Hom./hom.	GCN	89.58 ± 0.63	90.00 ± 0.38	75.92 ± 0.55	79.83 ± 0.49	12.75 ± 0.01	28.33 ± 0.03	64.17 ± 0.29	64.96 ± 0.38
	GAT	89.58 ± 0.06	90.00 ± 0.96	75.27 ± 0.59	78.38 ± 0.71	14.03 ± 0.05	16.55 ± 0.12	64.17 ± 0.27	64.96 ± 0.69
Het./hom.	NSD	**100.00 ± 0.23**	**100.00 ± 0.31**	73.60 ± 0.22	75.89 ± 0.49	33.65 ± 0.37	35.00 ± 0.21	72.30 ± 0.31	72.30 ± 0.10
	CSNN	**100.00 ± 0.39**	**100.00 ± 0.40**	76.64 ± 0.28	77.92 ± 0.07	35.15 ± 0.07	37.22 ± 0.13	74.11 ± 0.75	74.30 ± 0.80
Hom./het.	HAN	82.91 ± 0.61	85.00 ± 0.19	79.19 ± 0.82	82.22 ± 0.50	46.24 ± 0.06	45.02 ± 0.06	77.60 ± 0.43	77.50 ± 0.29
	PSHGCN	94.88 ± 0.10	95.00 ± 0.42	70.61 ± 0.13	72.32 ± 0.74	60.18 ± 0.24	58.33 ± 0.38	74.17 ± 0.34	74.96 ± 0.37
Het./het.	HDHGR	84.65 ± 0.71	85.00 ± 0.64	77.64 ± 0.83	81.25 ± 0.65	58.27 ± 0.21	58.33 ± 0.64	79.29 ± 0.61	79.32 ± 0.18
	Hetero2Net	89.58 ± 0.16	90.00 ± 0.62	74.66 ± 0.69	78.57 ± 0.43	61.53 ± 0.21	61.36 ± 0.49	63.67 ± 0.66	65.45 ± 0.38
	H2SGNN	94.88 ± 0.09	95.00 ± 0.35	70.61 ± 0.37	72.32 ± 0.14	62.18 ± 0.44	61.31 ± 0.45	64.17 ± 0.06	64.96 ± 0.44
	HetSheaf	**100.00 ± 0.00**	**100.00 ± 0.00**	80.56 ± 0.00	82.14 ± 0.00	62.35 ± 0.00	63.33 ± 0.00	79.46 ± 0.00	79.46 ± 0.00
	LatGRL	90.88 ± 0.99	91.01 ± 0.11	71.61 ± 0.20	72.32 ± 0.33	60.41 ± 0.09	60.03 ± 0.39	77.98 ± 0.11	78.10 ± 0.41
Ours	**H2S-NN**	**100.00 ± 0.35**	**100.00 ± 0.83**	**84.44 ± 0.21**	**86.23 ± 0.15**	**64.52 ± 0.56**	**65.10 ± 0.37**	**81.22 ± 0.06**	**81.52 ± 0.20**

Table 6: Performance on link prediction benchmarks reporting in AUPR and AUROC. Best values are bold.

		LastFM		PubMed		Amazon		YouTube	
Group	**Model**	AUPR	AUROC	AUPR	AUROC	AUPR	AUROC	AUPR	AUROC
Hom./hom.	GCN	97.06 ± 0.48	96.96 ± 0.41	96.05 ± 0.52	92.93 ± 0.36	92.40 ± 0.61	89.66 ± 0.73	89.30 ± 0.39	83.26 ± 0.52
	GAT	63.06 ± 0.27	51.14 ± 0.19	96.07 ± 0.43	93.27 ± 0.68	88.62 ± 0.34	86.10 ± 0.45	66.76 ± 0.31	58.40 ± 0.22
Het./hom.	NSD	97.16 ± 0.71	96.58 ± 0.65	96.41 ± 0.58	93.65 ± 0.74	92.63 ± 0.49	90.04 ± 0.38	89.32 ± 0.43	83.36 ± 0.57
	CSNN	97.99 ± 0.42	97.75 ± 0.39	96.14 ± 0.30	92.87 ± 0.29	92.10 ± 0.53	89.62 ± 0.41	88.64 ± 0.34	82.97 ± 0.48
Hom./het.	HAN	88.26 ± 0.37	83.32 ± 0.46	95.78 ± 0.49	93.62 ± 0.62	95.24 ± 0.56	93.64 ± 0.60	92.46 ± 0.51	90.10 ± 0.42
	PSHGCN	88.02 ± 0.44	80.55 ± 0.50	97.58 ± 0.35	93.27 ± 0.68	94.28 ± 0.46	92.34 ± 0.42	93.15 ± 0.63	90.66 ± 0.58
Het./het.	HDHGR	97.89 ± 0.51	97.93 ± 0.45	95.03 ± 0.77	93.15 ± 0.36	93.37 ± 0.40	91.42 ± 0.59	91.68 ± 0.65	88.33 ± 0.38
	Hetero2Net	97.58 ± 0.29	97.45 ± 0.61	94.66 ± 0.53	92.57 ± 0.47	94.71 ± 0.54	92.34 ± 0.42	90.67 ± 0.70	86.45 ± 0.41
	H2SGNN	97.88 ± 0.36	97.22 ± 0.68	95.22 ± 0.42	92.32 ± 0.55	93.18 ± 0.61	92.31 ± 0.50	90.17 ± 0.46	88.96 ± 0.64
	LatGRL	96.88 ± 0.62	96.01 ± 0.71	96.61 ± 0.48	92.32 ± 0.52	94.41 ± 0.43	92.03 ± 0.57	89.98 ± 0.35	88.10 ± 0.49
	HetSheaf	98.51 ± 0.00	98.00 ± 0.00	96.36 ± 0.00	93.00 ± 0.00	93.76 ± 0.00	91.00 ± 0.00	91.27 ± 0.00	88.00 ± 0.00
Ours	**H2S-NN**	**98.91 ± 0.37**	**98.72 ± 0.35**	**97.63 ± 0.41**	**96.48 ± 0.54**	**98.53 ± 0.56**	**98.33 ± 0.44**	**93.95 ± 0.47**	**91.81 ± 0.33**

automatically selected schemas yielded comparable results. The full AMPSS model achieved the best performance after introducing the instance-level gate β, reaching 96.01 Macro-F1 and 96.31 Micro-F1. These results suggest that schema-level selection alone cannot fully determine which meta-path information is useful. Instance-level weighting is also necessary to filter noisy or uninformative realizations of the selected schemas.

Table 7: Ablation Study of AMPSS on DBLP node classification.

Configuration	Macro-F1	Micro-F1
No meta-path cells	93.85	93.72
Predefined meta-paths	95.75	96.01
AMPSS, α only	95.88	95.92
Full AMPSS, $\alpha + \beta$	**96.01**	**96.31**

5.2 Ablation Study: Contribution of SheafConvPool

To evaluate the effectiveness of SheafConvPool, we compared the full H2S-NN on the ENZYMES graph-classification benchmark with four alternative graph-level readouts: sum pooling, mean pooling, and max pooling over the diffused node state $x^\star$, each followed by an MLP classifier, SheafPool [**?**], and SheafConvPool. As shown in Table **??**, the sum, mean, and max pooling performed comparably to one another. Replacing them with SheafPool raised Macro-F1 and Micro-F1 to 64.05/64.96, confirming that a gauge-invariant readout captures information that plain aggregation discards. SheafConvPool improved further still, reaching the best performance at 64.52 Macro-F1 and 65.10 Micro-F1. Unlike SheafPool, SheafConvPool does not whiten the per-node and per-edge Gram descriptors before scoring them; it considers not only node but also edge information when pooling. It also pools their attention-weighted mean together with the across-node and across-edge covariance of the projected descriptor. These results suggest that gauge invariance alone is not sufficient to fully summarize the node-sheaf state: retaining the second moment of the projected descriptor, rather than only its attention-weighted mean, provides an additional, complementary signal for graph-level classification.

Table 8: Ablation study of graph-level readouts on ENZYMES graph classification.

Readout	Macro-F1	Micro-F1
Sum + MLP	61.25	61.79
Mean + MLP	61.42	62.09
Max + MLP	61.38	61.62
SheafPool	64.05	64.96
SheafConvPool	**64.52**	**65.10**

6 Conclusion

This work addressed heterogeneous graph learning under heterophily, where connected nodes need not share a label or similar features and where relation types carry semantics that a shared update rule can blur. We introduced H2S-NN, which treats both properties as requirements on the operator rather than as separate pre- or post-processing steps: relation-conditioned node-to-edge maps compare each edge's endpoints in a shared local space instead of assuming they already agree, and selected meta-path instances are kept as face-like supports that shape diffusion over an explicit edge cochain, instead of being collapsed into induced node-to-node views. Matrix-free Hodge operators propagate node and edge cochains jointly from this geometry, AMPSS combines training-only schema preselection with differentiable schema- and instance-level refinement inside the upper edge Laplacian so that which meta-paths matter is learned rather than hand-specified, and SheafConvPool gives a finalized single-level, gauge-invariant readout over the learned node and edge states for graph classification.

The experiments support this framing. H2S-NN ranks first among the compared methods across every node-classification and graph-classification benchmark, including cases where it must outperform baselines that already combine heterogeneity- and heterophily-aware mechanisms. Link prediction is more mixed: H2S-NN leads on both LastFM metrics and PubMed AUPR, while stronger baselines remain on PubMed AUROC, Amazon, and YouTube, two single-node-type graphs where a type-aware node encoder has less structure to exploit. The ablation study shows that retaining meta-paths as instance-level, operator-level structure (AMPSS) improves prediction performance beyond either omitting them or fixing them to predefined schemas, and that SheafConvPool's gauge-invariant, second-moment pooling contributes materially to graph classification beyond gauge invariance alone. Altogether, these results indicate that jointly modeling type-aware relational structure and heterophily, rather than addressing either in isolation, and retaining meta-paths as higher-order operator-level structure rather than reducing them to node-to-node routes, are both beneficial for learning on heterogeneous heterophilic graphs.

A Hyperparameter Tuning

For all experiments, we used the same base hyperparameter values for the ACM, DBLP, and IMDB configurations. Dataset-specific overrides are stored in the resolved configuration of each run. The base hyperparameter values are summarized in Table **??**. Not yet ready

Table 9: Base hyperparameter values used in the ACM, DBLP, and IMDB configurations. Dataset-specific overrides are stored in the resolved configuration of each run.

B Benchmark Datasets Explanation

We evaluated the models on benchmark datasets covering node classification, link prediction, and graph classification. The node-classification datasets and most link-prediction datasets are obtained from the Heterogeneous Graph Benchmark (HGB) [**?**]. For graph classification, we used molecular and biological datasets from TUDataset [**?**].

B.1 Node Classification

DBLP. is a bibliographic network containing four types of entities: authors, papers, venues, and terms. These entities are connected through six relation types, including authorship, publication-venue, paper-term, and author-venue relations. The prediction target is the author node type, where each author is assigned to one of four research areas: database systems, data mining, artificial intelligence, or information retrieval.

IMDB. is a heterogeneous movie network composed of movies, directors, actors, and keywords. Its six edge types represent relations such as movie-director, movie-actor, and movie-keyword associations. The task is multi-label movie classification, in which a movie may belong to one or more of five genres: action, comedy, drama, romance, and thriller.

ACM. is an academic citation network with four node types: papers, authors, subjects, and terms. Its relations include paper citations, authorship, paper-subject associations, and paper-term associations. The objective is to classify papers into three research categories: database systems, wireless communication, and data mining.

B.2 Link Prediction

LastFM. is a music-oriented heterogeneous network containing users, artists, and tags. The graph includes user-artist, user-user, and artist-tag relations. The link-prediction task determines whether an interaction exists between a user and an artist.

MovieLens. represents a recommendation network with user and movie nodes. Edges encode user-movie rating interactions. The task is to predict whether a user is connected to a movie through a rating relation.

PubMed. is a biomedical knowledge graph containing genes, diseases, chemicals, and species. Ten relation types describe biomedical associations, including gene-disease, chemical-chemical, and species-disease relations. Evaluation focuses on predicting links between disease nodes.

YouTube. is a social interaction graph containing 2,000 user nodes connected by five distinct types of user interactions. The prediction task is to infer missing user-user links.

B.3 Graph Classification

MUTAG. contains 188 molecular graphs representing aromatic and heteroaromatic nitro compounds. Nodes correspond to seven atom types, while edges encode four bond types. Each graph is classified according to whether the corresponding compound exhibits a mutagenic effect.

PROTEINS. consists of 1,113 protein graphs. Nodes represent secondary-structure elements belonging to three types, and edges connect elements that are adjacent either in the amino-acid sequence or in three-dimensional space. The task is binary classification of proteins into enzyme and non-enzyme classes.

ENZYMES. contains 600 graph representations of protein tertiary structures collected from the BRENDA enzyme database. Its nodes describe three types of secondary-structure elements. Each graph is assigned to one of the six top-level Enzyme Commission classes.

NCI1. comprises 4,110 chemical compounds evaluated for their ability to inhibit the growth of human tumor cell lines. Molecular nodes represent 37 atom types. The graph-classification objective is to determine whether each compound exhibits anti-cancer activity.

C Additional Experiments: Node Clustering

For a more intuitively comparation and to ensure the performance of keeping locality after training. Although the main experiments evaluate supervised graph-learning tasks, we also used clustering as an additional diagnostic of representation quality. Following the clustering protocol used in the Heterogeneous Graph Attention Network paper [**?**], we ran k-means on the learned node embeddings after supervised training. We used DBLP with $k = 4$ and ACM with $k = 3$, matching the number of classes in each dataset, and set the maximum number of iterations to 300.

We compared H2S-NN against four groups of baselines: (i) homogeneous homophilic models, including GCN [**?**] and GAT [**?**]; (ii) homogeneous heterophilic models, including NSD [**?**] and CSNN [**?**]; (iii) heterogeneous homophilic models, including HAN [**?**] and PSHGCN [**?**]; and (iv) heterogeneous heterophilic models, including HDHGR [**?**], Hetero^{2}Net [**?**], H2SGNN [**?**], HetSheaf [**?**], and LatGRL [**?**].

We used Normalized Mutual Information (NMI) and Adjusted Rand Index (ARI) to evaluate clustering quality. We repeated the clustering step over five random seeds and report the mean and standard deviation. As shown in Table **??**, H2S-NN achieved the highest mean NMI and ARI on both DBLP and ACM. These results suggest that the embeddings learned by H2S-NN are more separable under this clustering diagnostic. The t-SNE visualizations in Figures **??** and **??** provide a qualitative view of the learned embeddings [**?**].



Figure 6: t-SNE visualization of learned ACM paper embeddings. Each point represents one paper, and color denotes its class.



Figure 7: t-SNE visualization of learned DBLP author embeddings. Each point represents one author, and color denotes its research area.

Table 10: Additional clustering-quality diagnostics on DBLP and ACM, reported as mean ± standard deviation over five runs. Best mean values are bold.

		DBLP		ACM	
Group	**Model**	NMI	ARI	NMI	ARI
Hom./hom.	GAT	71.50 ± 0.36	77.26 ± 0.48	57.22 ± 0.19	60.47 ± 0.31
Het./hom.	NSD	80.21 ± 0.62	84.04 ± 0.43	62.56 ± 0.37	65.02 ± 0.28
	CSNN	81.72 ± 0.39	84.25 ± 0.55	69.28 ± 0.45	71.10 ± 0.34
Hom./het.	HAN	79.12 ± 0.47	84.76 ± 0.58	60.63 ± 0.22	64.28 ± 0.41
	PSHGCN	80.38 ± 0.52	82.65 ± 0.36	71.89 ± 0.49	74.31 ± 0.57
Het./het.	Hetero2Net	80.54 ± 0.33	84.98 ± 0.46	70.14 ± 0.40	73.42 ± 0.51
	LatGRL	81.36 ± 0.56	86.07 ± 0.38	72.52 ± 0.61	76.75 ± 0.44
Ours	**H2S-NN**	**82.41 ± 0.35**	**86.52 ± 0.42**	**75.78 ± 0.37**	**78.29 ± 0.31**

D Additional Analysis: Diffusion Process of H2S-NN

We further inspected the training dynamics of H2S-NN on a controlled four-class `FakeHeteroDataset` from torch geometric library. The model was trained with the default synthetic-data settings, and the visualization includes only nodes with class labels. We recorded the classified-node embedding layout every five epochs. As shown in Figure **??**, at the beginning of training, nodes from different classes are strongly interleaved. As training proceeds, same-class nodes gradually move toward compact regions, and the four class-level groups become increasingly separated. This observation suggests that H2S-NN effectively learns to diffuse information across the heterogeneous graph structure, leading to improved class separability in the learned embeddings.

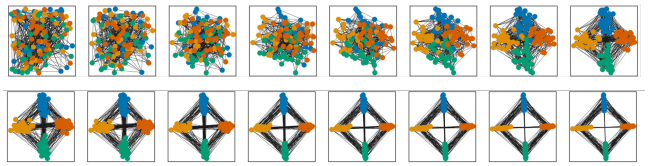


Figure 8: Training-time diffusion trajectory of H2S-NN on the four-class `FakeHeteroDataset`. Only labeled nodes are visualized, and consecutive panels are separated by five training epochs.

E Computational Efficiency

To have a better understanding of the computational cost, we conducted a big-O analysis of H2S-NN's, AMPSS's, and SheafConvPool's time and space complexity.

Let $N = |V|$ and $M = |E|$ be the number of nodes and directed edges, F the number of retained path instances, and $I = \sum_{f=1}^{F} \ell_f$ the total edge-face incidences (ℓ_f = length of instance f). Let d, d_r, d_h denote the stalk, relation-embedding, and sequence-encoder dimensions, and K the number of Hodge-diffusion steps. Because every edge on a retained path contributes its own edge-to-face restriction map, the upper Hodge term scales with I rather than F alone.

AMPSS preprocessing. Schema construction runs once, before training. With P label-propagation steps, C_y classes, C_Σ candidate schemas, S retained schemas, maximum schema length L, and Q_{type}, Q_{path} the number of typed and concrete path prefixes visited during enumeration and scoring, preprocessing costs

$$O\Big(P(M+N)C_y + Q_{\text{type}} + Q_{\text{path}} + \widehat{F}C_y + C_\Sigma SU + I\Big)$$

time and $O\big(M + NC_y + C_\Sigma(L+U) + I\big)$ space, where $\widehat{F}$ instances are sampled for scoring and U bounds a sampled endpoint set. In the worst case $Q_{\text{type}} = O(Tb_\Sigma^L)$ and $Q_{\text{path}} = O((C_\Sigma+S)N\bar{d}^L)$; in practice AMPSS caps C_Σ, S, and the number of materialized instances, so the realized cost is output-sensitive rather than exponential. At inference, the schema gates cost $O(S)$ per forward pass, and the instance gates (MLP width h_β) add $O\big(F(2d+d_r)h_\beta\big)$ time and $O(F)$ space.

H2S-NN. Table **??** summarizes the per-component cost of a forward pass, with node-to-edge and edge-to-face MLP widths h_e, h_f, and $q(d) = O(d^3)$ for the Householder parameterizations we implement ($O(d^2)$ for unrestricted dense maps).

Table 11: Per-component time and space complexity of the H2S-NN model. K is the number of diffusion steps

Component	Time	Space
Node-to-edge	$M\left[(2d+d_r)h_e + h_e d^2 + q(d)\right]$	Md^2
Edge-to-face	$I\left[(2d+d_r)h_f + h_f d_h + d_h^2 + d_h d^2\right]$	Id^2
Hodge diffusion	$K(M+I)d^2$	$(M+I)d^2$

Since node-to-edge and edge-to-face maps are cached and reused across all K diffusion steps, the full forward pass (excluding readout) costs

$$O\big(Nd_{\text{in}}d + (M+I)d^3 + K(M+I)d^2\big)$$

once hidden widths scale with d, linear in the number of edges and retained edge-face incidences for fixed d. Inference memory is $O\big((M+I)d^2 + (N+M)d + Id_h\big)$, dominated by the cached maps; training adds up to $O(K(N+M)d)$ for the diffusion trajectory.

SheafConvPool readout. With $r = 3$ node/edge views, projected-descriptor dimension f, pooling width h_p, batch size G, and C classes, the readout costs

$$O\big((N+M)[dr^2 + r^2h_p + h_pf + f^2] + G[h_pf^2 + h_pC]\big)$$

time and

$$O\big((N+M)(dr+f^2) + Gf^2\big)$$

space. With $r = 3$ and $f = 8$ fixed, this reduces to $O((N+M)d)$: a linear-time readout added on top of the model, with no dense Laplacian, node-node covariance, or coarsened graph ever built.

Furthermore, we reported measured runtime, parameter count, and memory on the ACM node classification task in Table **??**.

Table 12: Efficiency comparison on ACM. Best values are bold. Hetsheaf's code is not available, so we used the Hetsheaf-ensemble model reported results in the original paper.

Model	Runtime (s)	Parameters	Memory (MB)
R-GCN	**22.1**	43M	1,240
HAN	28.9	2.2M	1,780
HGT	1,025.8	7.2M	6,120
Sheaf-NSD	47.6	**155K**	**980**
Hetsheaf	55.7	174K	-
H2S-NN	72.1	517K	1120

H2S-NN is not the cheapest model in Table **??**: R-GCN has the lowest runtime, and Sheaf-NSD, which models neither relation types nor meta-paths, uses the fewest parameters and the least memory among the compared models. H2S-NN's runtime and parameter count are higher than Hetsheaf's, the closest heterogeneous sheaf baseline, which we attribute to the additional edge-to-face maps and AMPSS schema and instance gates that Hetsheaf does not have. Its memory footprint nonetheless stays within the same order of magnitude as the other compared models and well below the heterogeneous transformer baseline HGT, so retaining explicit face-level edge state does not come at a disproportionate memory cost.

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.